

Passive Security Analysis of Meshtastic Packet Captures

Ali Murtaza Bhutto
ORCID: 0009-0007-2787-943X
alibhutto101112@gmail.com

Abstract—Meshtastic is a community-developed LoRa mesh networking stack that ships in low-cost hardware and is widely deployed by hikers, ham operators, and emergency-services volunteers. The protocol’s defaults trade strong security guarantees for ease of bootstrapping: every device leaves the factory using one of seven publicly known modem presets and a one-byte default pre-shared key index, with the source and destination node identifiers transmitted in plaintext on every frame. This paper describes *meshtastic-security-audit*, an open-source toolkit that consumes a PCAP capture of Meshtastic frames and reports four classes of finding: use of the documented default channel, node-identifier enumeration exposure, plaintext payloads, and GPS coordinate leakage in unencrypted position frames. The toolkit’s parser, four detectors, reporter, and CLI run against a synthetic Scapy-crafted corpus of five frames and recover 9 of 9 expected detections (precision and recall both 1.000). The artefact is MIT-licensed and intended for authorised network audits.

Index Terms—Meshtastic, LoRa, LPWAN, mesh networking, security audit, packet analysis, AES-CTR, GPS leakage, passive analysis

I. INTRODUCTION

Low-power wide-area networks (LPWANs) have moved from industrial telemetry into general-purpose human-to-human communication, driven by hobbyist hardware (Heltec, T-Beam, RAK) and the open-source Meshtastic firmware. A typical out-of-the-box deployment routes text, position, and telemetry messages across kilometres of terrain at sub-watt transmit power, with no infrastructure beyond the nodes themselves. The convenience of that model rests on a small set of defaults: a fixed list of seven modem presets, a documented one-byte PSK index for the primary channel, and an unencrypted header that identifies the source and destination of every frame.

Each of those defaults is well known and intentional—the Meshtastic project documents them and they are necessary for interoperability between strangers in an ad-hoc mesh. They are also not what an operator running a sensitive deployment would choose if they understood the consequences. Two patterns recur in the practitioner discussions: an operator forgets to rotate the PSK on deployment, leaving traffic decodable by anyone with a Meshtastic node; and an operator enables position broadcasts on an unencrypted channel, broadcasting their location to every receiver in range.

This paper introduces a passive analysis toolkit that examines a PCAP capture of Meshtastic frames and reports the four most common classes of exposure. The toolkit is not a decryption attack on correctly configured devices and does not transmit, inject, or otherwise touch a live mesh.

a) Contribution.: (1) A Scapy-based parser that recovers the Meshtastic frame structure from a PCAP irrespective of the link-layer encapsulation used at capture time. (2) Four independent detectors, each emitting findings in a severity band that is consistent across the toolkit. (3) An HTML and JSON reporter that renders the findings for both humans and downstream tooling. (4) A synthetic, byte-level test corpus and an eval that measures the detectors against ground truth without requiring RF hardware.

II. BACKGROUND

a) Meshtastic frame structure.: A Meshtastic frame as it travels over the LoRa physical layer [2] consists of a 14- or 16-byte header (depending on firmware revision) followed by a length-variable payload. The header fields the toolkit consumes are: a 32-bit destination node identifier, a 32-bit source node identifier, a 32-bit packet identifier, a flags byte (carrying a 3-bit hop limit, a want-ack bit, and an encrypted-payload bit), and an 8-bit channel hash. The channel hash is the XOR of every byte of the channel name with every byte of the PSK [1].

b) Encryption.: When the encrypted-payload bit is set, the payload is encrypted with AES-256 in counter mode (CTR), using the packet identifier and source node id as the nonce material. CTR [3] is appropriate for short messages but provides no authentication; a separate authenticated mode would be preferable for a security-sensitive deployment, though that would break compatibility with the existing fleet.

c) Default channel.: For the primary channel, the firmware uses a one-byte PSK index of 0x01, which is expanded by the firmware into a 256-bit AES key. Any device using the same preset name and the same default index can decrypt the traffic. The toolkit detects this by computing the channel hash that would result for each documented preset name under the default index and checking whether the captured frames carry that hash.

d) Threats to confidentiality and location privacy.: The combination of plaintext source identifiers and unencrypted

position payloads is sufficient, on its own, to track a node across days of capture. The LPWAN security literature [4], [5] has flagged these classes of exposure as recurring failure modes across LoRaWAN, Sigfox, and Meshtastic deployments.

III. TOOLKIT ARCHITECTURE

A. Parser

`audit/pcap_parser.py` reads a PCAP via Scapy and, for each record, attempts to interpret the raw bytes as either the current 16-byte Meshtastic header or the legacy 14-byte variant. A small plausibility filter (non-zero source id, non-zero packet id, sane broadcast/unicast destination) screens out non-Meshtastic frames without raising. The result is a list of `MeshFrame` dataclasses with the header fields broken out and the encrypted payload preserved verbatim.

B. Detectors

Four detectors run independently against the parsed frames; each emits one or more `Finding` objects.

- **default_key** computes the channel hash for every documented preset name under the default PSK index and flags any frame whose channel byte matches. Severity: `high`.
- **node_enum** counts distinct source and destination node ids across the capture, emitting an `info` finding with the source inventory and a `medium` finding when more than one unicast destination is addressed.
- **unencrypted** reports two complementary signals: the encrypted bit being clear in the flags byte (`high`), and a payload whose first bytes parse as a plaintext Meshtastic Data protobuf (`critical`). The protobuf check is a stronger signal than the flag bit because it is content-based.
- **gps_leak** decodes any `Data` envelope on port 3 (`POSITION_APP`) and extracts the `latitude_i` and `longitude_i` fields. When a valid lat/lon pair is recovered, it emits a `critical` finding. The detector never attempts to decrypt encrypted payloads; an encrypted position frame produces no finding.

C. Reporter and CLI

`audit/reporter.py` renders the finding set as either an inline-styled HTML page (no external assets) or a JSON document with a stable summary block. The CLI (`cli.py`, Click-based) exposes `-pcap`, `-output`, `-format`, `-detector` (repeatable), and `-severity-threshold`.

IV. EVALUATION

A live regression run on the synthetic corpus in `tests/fixtures/test_packets.py` produces the results in Table I. The corpus contains five frames covering all four detector signals; ground-truth labels live in `eval/labelled_corpus.json` and the harness lives in `eval/run_eval.py`.

TABLE I
PER-DETECTOR CONFUSION-MATRIX METRICS ON THE SYNTHETIC LABELLED CORPUS (5 FRAMES). THE CORPUS IS INTENTIONALLY SMALL AND CONTROLLED; PER-DETECTOR GROUND TRUTH IS THE UNION OF DETECTOR SIGNALS EACH FRAME IS CONSTRUCTED TO EXHIBIT. NUMBERS REPRODUCE BY RUNNING `PYTHON EVAL/RUN_EVAL.PY`.

Detector	TP	FP	FN	P	R	F ₁
<code>default_key</code>	1	0	0	1.000	1.000	1.000
<code>node_enum</code>	5	0	0	1.000	1.000	1.000
<code>unencrypted</code>	2	0	0	1.000	1.000	1.000
<code>gps_leak</code>	1	0	0	1.000	1.000	1.000
Aggregate	9	0	0	1.000	1.000	1.000

The pytest suite (`tests/`) carries 23 tests: 7 for the parser, 8 for the detectors, 4 for the reporter (HTML, JSON, severity ordering, empty-input handling), and 4 end-to-end CLI tests via the Click `CliRunner`. The full suite executes in approximately 1.05 seconds on the reference host.

A. DevSecOps gates

The release passes three gates:

- **Bandit** (medium severity and above) reports zero findings across 840 lines of Python.
- **pip-audit** reports no known vulnerabilities in the installed dependency closure after pinning `pytest` $\geq 9.0.3$ and upgrading `pip`.
- **Semgrep** with the `p/python` and `p/security-audit` rule packs exits 0.

Raw scan outputs are committed under `results/security_scan.md`.

V. THREAT MODEL

The toolkit is read-only. It produces no RF traffic, opens no network sockets, and does not attempt to decrypt encrypted payloads. The attack surface is therefore the PCAP parser itself: a malformed or adversarial PCAP could crash the parser, but the plausibility filter and Scapy's own input handling bound the impact to a Python exception. The CLI accepts only local file paths.

The toolkit is intentionally narrow about what it does *not* attempt:

- No active RF capture. The operator must produce the PCAP via an SDR pipeline or the firmware's debug serial sniffer.
- No decryption attack on correctly configured channels. A user who has rotated the PSK is outside the toolkit's reach; that is the intended outcome.
- No node identity resolution. The output stays at the `!xxxxxxx` layer; mapping that identifier to a real human is an operator decision and is governed by the authorisation chain.

VI. CONCLUSION

meshtastic-security-audit is a small, focused toolkit that operationalises four well-understood Meshtastic-specific

exposures into a single command-line audit. The contribution is not a new attack; the defaults this toolkit flags have been documented by the Meshtastic project for years [1] and are unavoidable consequences of the protocol's bootstrapping model. The contribution is reproducible, measured, low-effort auditing of the protocol surface, so an operator deploying Meshtastic in a sensitive environment can answer "what would an observer with a receiver in range see?" before deploying rather than after.

REFERENCES

- [1] Meshtastic Project, "Meshtastic protocol documentation," 2024. <https://meshtastic.org/docs/overview/> and the protobuf definitions at <https://github.com/meshtastic/protobufs>.
- [2] Semtech Corporation, "LoRa modulation basics," Application Note AN1200.22, 2015. <https://www.semtech.com/products/wireless-rf/lora-connect>
- [3] M. Dworkin, "Recommendation for block cipher modes of operation: Methods and techniques," National Institute of Standards and Technology, NIST Special Publication 800-38A, Dec. 2001.
- [4] F. Adelantado, X. Vilajosana, P. Tuset-Peiro, B. Martinez, J. Melia-Segui, and T. Watteyne, "Understanding the limits of LoRaWAN," *IEEE Communications Magazine*, vol. 55, no. 9, pp. 34–40, Sep. 2017.
- [5] European Union Agency for Cybersecurity (ENISA), "ENISA threat landscape 2024," Sep. 2024. <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2024>